SOFTWARE ENGINEERING 381 - SEN381

INTEGRATED TEAM SOFTWARE ENGINEERING PROJECT

CIVICCONNECT

Community Service Request Management Platform

Module	Software Engineering 381 - SEN381
NQF Level	8
Project Mode	Team Project
Team Size	3 students
Project Structure	4 assessed milestones, including final product and engineering defence
Project Contribution	Project component assessed out of 100 and weighted 50% of the final practical examination
Academic Year	2026

Central Project Principle

Students are assessed on whether they can engineer, control, justify, verify and defend a software product across its lifecycle. Working code is necessary, but working code alone is not sufficient evidence of Software Engineering competence.

Table of Contents

Document Control

Version	Date	Purpose	Approval / Owner
1.1	2026	Reviewed student-release brief: acronym clarity, technology-selection responsibility, uniform milestone raw/weight representation, prior-learning progression, presentation requirements and final disclaimer	SEN381 Teaching Team

Acronyms and Abbreviations

The following acronyms are used throughout this brief. Students are expected to understand both the full term and its Software Engineering purpose.

Acronym	Full name
ADR	Architecture Decision Record
AI	Artificial Intelligence
API	Application Programming Interface
BC	Belgium Campus ITversity
CI	Continuous Integration
CI/CD	Continuous Integration / Continuous Delivery or Deployment
NFR	Non-Functional Requirement
PED	Project Engineering Document
PR	Pull Request
RBAC	Role-Based Access Control
RTM	Requirements Traceability Matrix
UI	User Interface
URL	Uniform Resource Locator

How to Use This Brief

This Master Project Brief is the single source of truth for project-wide rules, standards, evidence requirements, team accountability and assessment expectations. The four milestone briefs will be deliberately shorter and will specify only the phase-specific work, deliverables and presentation requirements. Where a milestone refers to a project-wide standard, students must apply the relevant requirement in this brief.

Important

The standards in this brief are mandatory engineering controls, not optional recommendations. Failure to apply a required control can reduce marks even where the software appears to function.

Project Assessment Architecture

Project Phase	Primary Focus	Weighted Team Contribution	Weighted Individual Contribution
Milestone 1	Engineering Foundation & Requirements Baseline	10	5
Milestone 2	Architecture, Design & Engineering Decisions	15	10
Milestone 3	Controlled Construction, Integration, Quality & Release Readiness	18	12
Milestone 4	Final Product, Project Success & Engineering Defence	15	15

Weighted Project Mark: 100 marks. The project component contributes 50% of the final practical examination. Milestones may be assessed on a larger raw-mark scale to allow fair and practical mark discrimination, then converted to their approved project weighting. The raw scale is an assessment instrument and does not change the milestone's weighted contribution. Each milestone combines shared team engineering evidence with individually assessed presentation and engineering-defence evidence. Students within the same team may therefore receive different milestone and project marks.

1. Project Purpose

The SEN381 project is designed to consolidate prior programming, database, web, analysis and design knowledge within a contemporary Software Engineering environment. Students are not being assessed merely on whether they can implement features. They must demonstrate that they can establish a controlled engineering foundation, make evidence-based decisions, manage change, collaborate securely, verify quality, prepare software for release, operate within constraints and explain the downstream consequences of earlier decisions.

- Analyze stakeholder needs, business value and project/product scope.
- Work within schedule, cost, resource, quality and security constraints.
- Create clear, testable and traceable requirements and acceptance criteria.
- Identify, assess and manage engineering risks and assumptions.
- Evaluate architecture, design, technology, data and deployment alternatives.
- Use professional configuration management, GitHub governance and peer review.
- Use AI responsibly as an engineering assistant while retaining human verification and accountability.
- Build, integrate and verify software using automated build, testing and quality/security checks.
- Control change through formal impact analysis and baseline management.
- Produce measurable quality evidence rather than unsupported quality claims.
- Prepare and deploy software through controlled environments, including staging and production.

- Evaluate operational readiness, observability, rollback and recovery.
- Assess stakeholder satisfaction and project success against the original constraints.
- Reflect on technical debt, maintenance needs and lessons from engineering decisions.

2. Project Scenario: CivicConnect

A community-focused organisation currently manages service requests through a fragmented combination of email, telephone calls, WhatsApp messages, spreadsheets and paper-based records. Service requests include matters such as facility faults, damaged equipment, security concerns, IT support, maintenance issues, lost property and other operational requests.

The current process creates recurring operational and information-management problems:

- Requests may be duplicated, overlooked, incorrectly assigned or lost between channels.
- Requesters have limited visibility of whether a request was received, assigned, delayed, resolved or closed.
- Staff have difficulty prioritising requests, identifying ownership and coordinating work.
- There is weak accountability for changes to request status and actions taken.
- Management has limited reliable information about outstanding, overdue and resolved work.
- Reporting is manual, inconsistent and difficult to audit.
- Sensitive request information may be handled inconsistently across informal communication channels.
- There is no single controlled record of the lifecycle of a service request.

2.1 Business Need

The organisation needs a controlled digital platform that provides a reliable, traceable and usable way to submit, manage, monitor and report on service requests. The solution must improve visibility and accountability without creating an unsustainable technical, operational or financial burden.

2.2 The Engineering Challenge

Do not treat this as a coding specification

The brief defines the problem and minimum business capabilities. The team must still perform requirements engineering, define measurable quality expectations, make architecture and technology decisions, design interfaces and persistence, manage risk, control change, test, deploy and defend the resulting solution.

3. Minimum Business Capabilities

The final system must, at minimum, support the following broad business capabilities. These are not complete detailed requirements; the team must derive and baseline project requirements during Milestone 1.

Requester capabilities

- Submit a new service request with appropriate information.
- Categorise a request using a controlled category mechanism.
- View the current status of submitted requests.
- View a history/list of previously submitted requests.
- Receive meaningful feedback when a request is accepted, rejected, updated or completed.

Staff capabilities

- View service requests relevant to authorised staff.
- Search, filter or sort requests using useful criteria.
- View full request details.
- Assign or accept responsibility for a request.
- Update request status through controlled transitions.
- Record relevant actions, comments or resolution information.
- Resolve or close requests where authorised.

Management / oversight capabilities

- View useful service activity information.
- Identify open, overdue, resolved and closed requests.
- View request information by category/status or other justified dimensions.
- Access enough information to support accountability and service-performance analysis.

3.1 Additional Features

Teams may propose additional functionality only when it is justified by stakeholder value and the effect on scope, schedule, cost, quality, security and risk has been considered. Additional features do not automatically attract additional marks.

Scope discipline

Every additional feature creates obligations to specify, design, secure, implement, test, document, deploy and maintain it. A smaller, controlled, high-quality solution may demonstrate stronger Software Engineering than a larger unfinished solution.

4. Project Constraints and Success Boundaries

Teams must engineer within the following explicit constraints. These constraints are deliberately part of the assessment and must influence decisions.

Constraint	Minimum Expectation	How It Will Be Assessed
Team size	Exactly 3 students per project team.	Contribution history and individual milestone defence.
Schedule	The project must progress through four formal milestones within the SEN381 delivery period.	Milestone readiness, variance explanations, scope decisions and evidence of schedule impact.
Cost	Teams should prefer free or low-cost services where practical, but must identify limitations and likely operational cost beyond the educational context.	Decision records, platform research evidence, final cost review.
Scope	Committed scope must be baselined and controlled; uncontrolled scope creep is not acceptable.	Scope baseline, change records, final delivered-vs-approved scope evaluation.
Quality	Quality attributes must be defined and later supported by measurable evidence.	NFRs, acceptance criteria, test evidence, quality gates and final assurance.
Security	Security is a lifecycle-wide responsibility, not a final add-on.	Security requirements, repository controls, threat review, scans, secrets handling, deployment controls.
Technology	No stack, architecture or platform is prescribed; decisions must be justified against requirements and constraints.	ADRs, decision matrices, trade-off analysis, implementation consequences.

5. Definition of Project Success

Project success will be evaluated against the original stakeholder expectations and project constraints, not simply by whether the final URL opens or the user interface appears polished.

Stakeholder Value →	Agreed Scope →	Schedule →	Cost/Resources →	Quality →	Security →	Risk →	Deployability →	Operational Readiness

At the final defence, teams must show which stakeholder expectations were satisfied, partially satisfied, changed, rejected or not achieved, and support each conclusion with traceable evidence.

6. One Project, One Evolving Engineering Record

The team must maintain a single Project Engineering Document (PED) that evolves through all project phases. The PED must not be recreated as four unrelated reports.

Version	Project Phase	Minimum New/Updated Content

v1.0	Milestone 1	Problem, stakeholders, scope, requirements, constraints, risk, process, governance, baseline.
v2.0	Milestone 2	Architecture, design, technology, data, UI, APIs, ADRs, trade-offs, updated risks/traceability.
v3.0	Milestone 3	Change impact, construction/integration evidence, CI, testing, quality, security, staging/release readiness.
v4.0	Milestone 4	Final success evaluation, production/deployment evidence, stakeholder validation, decision consequences, debt/evolution.

6.1 PED Quality Standard

- Use clear section numbering and a professional structure.
- Maintain version history, authorship, review and approval information.
- Use consistent identifiers for requirements, risks, decisions, ADRs, changes, tests and defects.
- Do not silently overwrite baselined content. Preserve history and link changes to approved change records.
- Avoid screenshots when stronger live or traceable evidence exists.
- Use diagrams/tables where they improve engineering understanding; every diagram must have a clear purpose.
- Ensure statements of quality, security, performance or readiness are supported by measurable evidence.
- Use concise professional writing. Length alone does not demonstrate quality.
- All major claims presented to assessors must be traceable to the PED, repository, pipeline, tests, deployment evidence or another controlled project artefact.

7. Project Artefact and Evidence Standard

Every required artefact will be assessed at three levels:

Level	Question	What Strong Evidence Looks Like
Artefact	Does the required output exist and meet its purpose?	Complete, current, structured, correctly identified and integrated with the project.
Evidence	Does the artefact support the team's claim?	Traceable links to requirements, repository history, tests, pipeline results, deployment or stakeholder evidence.
Understanding	Can the student explain why it matters to Software Engineering?	Explains trade-offs, risks, downstream effects, limitations and why the chosen control/technique was appropriate.

Examples

A green CI pipeline is not enough: students must explain what the pipeline verifies and what it cannot prove. An ADR is not enough: students must defend the alternatives, decision, rationale, risk and consequence. A test report is not enough: students must explain which requirement/risk it addresses and why the technique is appropriate.

7.1 Team Formation and Registration

The CivicConnect project is completed in teams of three students. Team formation is treated as the first professional collaboration responsibility of the project.

- Students may establish their own team of exactly three members from the relevant SEN381 cohort.
- Self-selected teams must be finalised and communicated/registered with the lecturer by end of day on Monday, 31 August 2026.
- A student may belong to one project team only. Team membership is not confirmed until all three members are identified and the team is registered.
- Students who have not joined and registered a complete team by the deadline will be placed into teams automatically after the deadline.
- Automatically allocated teams are official project teams and are subject to the same project, accountability, contribution and assessment requirements as self-selected teams.
- Students should not delay Assignment 1 or Milestone 1 preparation while waiting for preferred team arrangements. Once teams are confirmed, all controlled project work must be attributable to the registered team.
- Any exceptional request to change a registered team after the deadline must be raised with the lecturer and must not be assumed to be approved. Project evidence, accountability and assessment integrity will guide any decision.

8. Team Engineering and Individual Accountability

Software Engineering is collaborative, but assessment of competence is individual. Each team submits shared artefacts, while each student is separately assessed during every milestone presentation.

- All three students must understand the complete project, not only the tasks they personally implemented.
- Roles may be assigned, but roles do not remove collective responsibility for the engineered product.
- Repository, issue, review and document history must show authentic progressive participation.
- A student cannot receive full individual marks by relying on another team member to answer questions.
- Individual marks may differ significantly even where the team artefact score is shared.

8.1 Minimum Individual Evidence

- Meaningful commits attributable to the student.
- Ownership of project issues/tasks.
- Pull Requests authored by the student.
- Pull Requests meaningfully reviewed by the student.
- Contribution to controlled documentation/decision records.
- Ability to trace at least one requirement through design, implementation and verification.
- Ability to explain at least one engineering decision and its consequences.

- Ability to explain relevant AI-assisted work and how it was verified.

9. GitHub Governance and Configuration Management Standard

GitHub is part of the project's engineering control environment, not merely a file-storage service.

Control	Mandatory Expectation
Repository	One controlled team repository unless a different structure is justified and approved.
Main branch	Protected and treated as the controlled product state.
Direct development on main	Not permitted for substantive controlled changes.
Pull Requests	Required for substantive changes entering main.
Approval	Minimum of TWO approvals from team members other than the author.
Self-approval	Not accepted.
Review quality	Approval must reflect meaningful review; rubber-stamping may receive no credit.
Issues/tasks	Meaningful engineering work should be represented in the project board/backlog and linked where practical.
Secrets	Passwords, API keys, tokens, private keys and confidential credentials must not be committed.
History	Commits, PRs, reviews, merges and changes must be progressive and authentic.

9.1 Meaningful Review

A strong review may evaluate:

- Alignment with requirements and acceptance criteria.
- Correctness and design consistency.
- Maintainability and technical debt.
- Security and privacy implications.
- Tests and regression implications.
- Dependency changes.
- Documentation/traceability impact.
- Whether the change should be accepted into the controlled baseline.

A change may follow this evidence path:

Review Comment →	Author Response →	Correction →	Re-review →	Approval →	Merge

10. Responsible Artificial Intelligence (AI) Engineering Standard

AI is integrated throughout SEN381 as an engineering assistant. AI output is not authoritative evidence and does not transfer accountability away from the student or team.

- AI may assist analysis, research, architecture comparison, design, coding, testing, debugging, review, documentation and operational analysis where permitted.
- Material AI contributions must be recorded in the AI Usage Register.
- Students must verify important AI claims against credible evidence and/or technical tests.
- AI-generated code is subject to the same branch, build, test, security and peer-review controls as human-written code.
- Students must not expose credentials, confidential material or inappropriate personal/sensitive data to external AI systems.
- Students must be able to explain, defend and modify any AI-assisted artefact they submit.
- The statement 'AI generated it' is never an acceptable engineering defence.

10.1 AI Usage Register Minimum Fields

Date	Student	Tool	Engineering task	AI contribution	Verification	Decision	Issues found

11. Requirements, Traceability and Baseline Standard

- Functional requirements must use unique identifiers (for example FR-001).
- Non-functional/quality requirements must be specific and measurable where practical.
- Important requirements must have acceptance criteria.
- Requirements must identify source/stakeholder and priority.
- The team must maintain one evolving Requirements Traceability Matrix (RTM).
- Baselined requirements may not be silently edited after sign-off.
- Approved changes must update the RTM and affected engineering artefacts.

11.1 Expected Final Traceability

Stakeholder/Source →	Requirement →	Design/Architecture →	Issue/PR →	Implementation →	Test →	Acceptance/Release Evidence

12. Risk, Assumptions and Constraint Management

The Risk Register is a live engineering artefact and must be reviewed at every milestone.

Risk ID	Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status

- Risks must be specific enough to manage; vague entries such as 'coding problems' are insufficient.
- Assumptions that could materially affect the project should be considered for risk treatment.
- When a risk materialises, it becomes an issue and should be connected to actions/decisions.
- Risks should include requirements, people/skills, technology, security, integration, deployment, cost, schedule, quality, dependencies and AI where relevant.

13. Engineering Decision and ADR Standard

Significant decisions must be recorded so that later consequences can be evaluated.

Decision ID	Context	Constraints	Alternatives	Decision	Rationale	Trade-offs	Risks	Evidence	Later consequence

Major architecture, technology, persistence, integration, authentication, deployment and infrastructure choices should use an Architecture Decision Record (ADR) or equivalent structured decision record.

14. Change Management Standard

After a baseline is approved, changes must be controlled rather than silently absorbed.

Change Request →	Impact Analysis →	Decision →	Authorise →	Implement →	Verify →	Update Baseline

Impact analysis must consider, where relevant:

- Requirements and acceptance criteria.
- Architecture and design.
- User interface and information architecture.
- Data/persistence and migration.
- API/interface contracts and consumers.
- Security and privacy.
- Scope, schedule, cost and resources.
- Testing and regression.

- Deployment and operations.
- Risk and technical debt.

15. Quality Engineering and Evidence Standard

Quality claims must be supported by evidence that is traceable to requirements, quality attributes and risks.

- Distinguish Quality Assurance (process controls) from Quality Control (product detection/measurement).
- Use risk-based testing to prioritise important behaviour and failure conditions.
- Use appropriate white-box, black-box, integration, system, regression and performance techniques.
- Interpret coverage carefully; high coverage does not prove correctness or quality.
- Maintain a defect register with severity/priority and disposition.
- Quality gates must define what evidence is required before merge and before release.
- AI-generated tests must be independently evaluated for missing cases and incorrect assumptions.

16. Security Engineering Standard

Security must be treated as a lifecycle-wide engineering responsibility.

Requirements →	Architecture →	Design →	Code →	Repository/CI →	Testing →	Deployment →	Operations

- Identify security requirements and sensitive-data implications early.
- Review trust boundaries, authentication, authorisation and least privilege.
- Protect secrets through approved configuration mechanisms.
- Review dependencies and vulnerability findings.
- Use static/security checks where appropriate.
- Verify security-relevant behaviour with tests/evidence.
- Record residual security risks rather than claiming the system is 'secure'.

17. Environments, Deployment and Operations Standard

Teams must distinguish development, test, staging and production environments and understand why success in one does not guarantee success in another.

Environment	Primary Purpose	Typical Differences / Concerns
Development	Frequent local/team development and experimentation.	Developer configuration, mock/test services, rapid change.
Test	Controlled verification.	Test data, automated suites, integration checks.
Staging	Production-like validation of a release	Production-like configuration, deployment path,

	candidate.	database/runtime compatibility, final checks.
Production	Real operation for intended users.	Real data, real security exposure, availability, monitoring, cost and recovery responsibility.

- Environment-specific configuration and secrets must be controlled.
- Deployment and release are not identical; teams must identify how release approval occurs.
- Rollback/recovery must be considered before production release.
- Operational readiness includes logging, monitoring, health checks, failure detection and incident response.
- The final deployment decision must revisit earlier assumptions, compatibility, cost, scalability and operational risk.

18. Cost, Schedule and Resource Accountability

Cost, schedule and team capability are engineering constraints. Teams must demonstrate how these constraints influenced scope, architecture, technology, testing and deployment decisions.

- Track major schedule variance and explain engineering causes, not only administrative delay.
- When schedule pressure appears, do not silently reduce testing/security. Record the trade-off and obtain appropriate approval where needed.
- Document free-tier limits and likely operational costs for the selected platform/services.
- Consider learning-curve risk when selecting unfamiliar technologies.
- Final project success must evaluate actual delivery against these constraints.

18.1 Technology Selection and Environment Responsibility

Technology selection is an assessed Software Engineering decision, not a preference exercise. Teams must investigate whether the proposed stack, libraries, services, development tools and deployment platform are appropriate for the approved requirements, constraints and the team's actual capability before committing to them.

The team must be able to justify its technology choices using evidence and explain the trade-offs and risks created by those choices. At minimum, the selection should consider:

- Requirement and architecture fit
- Team capability and realistic learning curve
- Availability and compatibility within the available development environment
- Security ecosystem, dependency maturity and support
- Maintainability and future change
- Testing, automation and tooling support
- Deployment and operational compatibility
- Development and likely operational cost

- Vendor/platform limitations and lock-in risk
- Consequences if the selected technology becomes unavailable or unsuitable

Evidence may include a weighted decision matrix, small technical experiments or proof-of-concepts, official documentation, cost/platform research and an Architecture Decision Record (ADR). The final choice remains the responsibility of the team.

19. Milestone Presentation and Defence Standard

Every milestone includes a formal presentation and engineering defence. The presentation is not a summary of the document: it is an evidence-based engineering review in which the team demonstrates controlled artefacts and each student demonstrates individual competence. Presentation skills are assessed explicitly as professional engineering communication, including logical structure, clarity and conciseness, effective use of artefacts/visual evidence, delivery and engagement, timing, transitions and professional conduct. Presentation quality does not replace technical evidence or Software Engineering understanding.

19.1 Required Presentation Behaviour

- Use live artefacts where practical: repository, PRs, pipeline, test reports, staging/production evidence, logs and traceability.
- Slides may guide the presentation but must not replace the engineering evidence.
- Demonstrate the artefact, explain why it exists, show how it links to the project and identify its downstream effect.
- All three students must participate in every assessed milestone presentation and individual engineering defence, regardless of the type or amount of contribution made to the shared milestone artefacts.

Presentation is compulsory assessment evidence. A student who does not present receives no mark for the presentation/individual milestone assessment component, except where an accepted exceptional circumstance applies and an alternative presentation has been formally arranged with the lecturer.

- Assessors may direct questions to any individual, not only the person who presented the relevant slide.
- Students must be able to identify limitations, unresolved risks and evidence gaps honestly.
- Presentation skill and engineering understanding are assessed separately: polished delivery cannot compensate for weak evidence or weak technical understanding, and strong technical work must still be communicated professionally.

19.2 Standard Presentation Evidence Pattern

The following pattern provides a consistent structure for milestone presentations. Students are expected to demonstrate rather than merely describe their engineering work. Each presentation should connect the artefact being shown to verifiable evidence, its Software Engineering purpose, the judgement and trade-offs behind it, and the downstream consequences for the project. Assessors may use the same pattern to guide individual questioning.

Show Artefact →	Show Evidence →	Explain SE Purpose →	Explain Trade-off/Risk →	Explain Downstream Effect →	Answer Individual Question

19.3 Typical Individual Question Categories

- Requirements/scope: Which requirement changed, and what did that change affect?
- Architecture: Which quality attribute or constraint drove this decision?
- Technology: Why was this stack/platform appropriate for this team and schedule?
- Design: Where is coupling highest, and what future problem could it create?
- GitHub/review: Show a PR you reviewed. What engineering issue did you identify?
- Quality: What does this test result prove, and what does it not prove?
- Security: What residual security risk remains and why is it acceptable/not acceptable?
- Deployment: Why should staging resemble production, and what can still differ?
- Operations: How would you know the system is degrading before users complain?
- AI: Show one material AI output that was modified/rejected and explain why.
- Technical debt: Which debt item would you address first if given two additional weeks?
- Evolution: Which part of the system would be hardest to change for a new stakeholder need?

20. Project Milestone Structure

Detailed milestone briefs will specify phase-specific requirements. The following overview defines the minimum project direction and assessment focus.

20.1 Milestone 1 — Engineering Foundation & Requirements Baseline (Raw: 50 marks; Project weighting: 15 marks)

Central question: What are we engineering, why, for whom, and within what constraints?
Assessment structure: 30 raw marks for shared team artefacts/evidence and 20 raw marks for individual examination evidence. The individual component comprises Presentation Skill & Professional Communication (5) and Individual Engineering Defence & Software Engineering Understanding (15). The raw /50 result is converted to the approved M1 project weighting.

Prior learning applied: Systems Analysis and Design requirements elicitation/modelling, stakeholder and scope analysis, introductory project planning, programming logic and basic source-control/team practices.

SEN381 progression: students move from analysing a proposed system to establishing an auditable engineering baseline. Prior artefacts are not repeated mechanically; they are strengthened through measurable acceptance criteria, traceability, risk/constraint reasoning, controlled collaboration, responsible AI use, baseline sign-off and forward lifecycle thinking.

- Problem statement, business need and stakeholder analysis.

- Scope baseline: in scope, out of scope and future/optional scope.
- Functional and non-functional requirements.
- Acceptance criteria.
- Assumptions and constraints: scope, schedule, cost, resources, quality and security.
- Initial RTM.
- Initial Risk Register.
- Initial Engineering Decision Log.
- Process/project approach and team working agreement.
- GitHub repository and mandatory governance controls.
- AI Usage Register.
- Initial deployment/operational considerations.
- PED v1.0 baseline and sign-off.

Assessment component	Marks	Purpose
Team artefacts/evidence	30	Quality, control, traceability and readiness of the M1 engineering baseline.
Presentation skill & professional communication	5	Individual professional engineering communication: structure, clarity, evidence use, delivery, timing and conduct.
Individual engineering defence & SE understanding	15	Individual command of evidence, principles/standards, traceability, trade-offs, downstream consequences and accountability.

20.2 Milestone 2 — Architecture, Design & Engineering Decisions (Raw: 50 marks; Project weighting: 25 marks)

Central question: How should we engineer the solution, and why?

Assessment structure: 30 raw marks for shared team artefacts/evidence and 20 raw marks for individual examination evidence. The raw /50 result is converted to the approved M2 project weighting of 25 marks.

Prior learning applied: Systems Analysis and Design modelling, database/data design, user-interface design, programming and Object-Oriented Programming principles, web/application development and earlier technology exposure.

SEN381 progression: students must justify architecture, technology and design decisions against quality attributes, constraints, security, cost, deployment and maintainability. Existing design/programming knowledge becomes evidence for engineering trade-offs rather than an exercise in reproducing diagrams or selecting familiar tools.

- Architecturally significant requirements and quality drivers.
- Architecture alternatives and justified selection.

- Architecture diagrams and ADRs.
- Technology-stack decision with security, cost, schedule, team-capability and deployment implications.
- Data/persistence design.
- Information architecture and user-interface/interaction design.
- Wireframes/prototypes and usability/accessibility rationale.
- API contracts and integration design.
- Relevant design principles/patterns and explicit avoidance of unnecessary complexity.
- Initial working/design prototype where required by the milestone brief.
- Updated RTM, Risk Register, Decision Log and PED v2.0.
- Evidence of responsible AI-assisted research/design where used.

Assessment component	Marks	Purpose
Team artefacts/evidence	15	Depth and coherence of architecture/design decisions.
Individual defence	10	Ability to justify alternatives, trade-offs, risks and consequences.

20.3 Milestone 3 — Controlled Construction, Integration, Quality & Release Readiness (Raw: 50 marks; Project weighting: 30 marks)

Central question: Can the team safely construct, change, integrate, verify and prepare the system for release?

Assessment structure: 30 raw marks for shared team artefacts/evidence and 20 raw marks for individual examination evidence. The raw /50 result is converted to the approved M3 project weighting of 30 marks.

Prior learning applied: programming and Object-Oriented Programming, Web Programming/application development, database integration, debugging, testing fundamentals and version-control experience.

SEN381 progression: individual coding skills are integrated into controlled team engineering. Students must demonstrate configuration management, peer review, Continuous Integration (CI), automated verification, quality/security evidence, controlled change, staging deployment and traceability from requirement through implementation and test.

- Substantial working implementation.
- Formal lecturer/client change request and impact analysis.
- Controlled implementation of approved change.
- Feature branches, meaningful commits and traceable Pull Requests.
- Two-reviewer approval evidence and meaningful review/rework.
- Automated build and dependency restoration.
- Automated unit/integration/regression tests.
- Static analysis and dependency/vulnerability checks.
- Quality gates and build/test failure handling.
- Requirements-to-code-to-test traceability.
- Quality/test strategy and evidence.

- White-box, black-box, integration/system and performance evidence as appropriate.
- Defect register and residual risk.
- Staging deployment and environment-parity evidence.
- Configuration/secrets handling.
- Security assurance/threat review and mitigations.
- Production-readiness review, rollback planning and operational monitoring concept.
- Updated technical-debt register, Risk Register, Decision Log/ADRs, RTM and PED v3.0.

Assessment component	Marks	Purpose
Team artefacts/evidence	18	Demonstrated controlled engineering workflow and release-readiness evidence.
Individual defence	12	Understanding of change, CI, quality, security, environments and project consequences.

20.4 Milestone 4 — Final Product, Project Success & Engineering Defence (Raw: 50 marks; Project weighting: 30 marks)

Central question: Did the project succeed, and can each student defend the engineering evidence and consequences?

Assessment structure: 25 raw marks for shared team artefacts/evidence and 25 raw marks for individual examination evidence. The raw /50 result is converted to the approved M4 project weighting of 30 marks.

Prior learning applied: the complete body of prior Systems Analysis and Design, programming, database, web/application development, testing and project-work knowledge, together with evidence accumulated in Milestones 1–3.

SEN381 progression: students evaluate the software as an engineered product rather than only demonstrate functionality. They must defend stakeholder value, baseline performance, quality, security, deployment/operations, technical debt, maintainability, project constraints and the actual consequences of earlier engineering decisions.

- Final working product and production/release evidence.
- Stakeholder validation against original needs and acceptance criteria.
- Delivered scope compared with approved baseline.
- Schedule performance and variance analysis.
- Cost/resource evaluation and likely operational cost.
- Quality and security evidence summary.
- Deployment, rollback, observability and operational readiness evidence.
- Final Risk Register, technical-debt/evolution view and known limitations.
- Final RTM and PED v4.0.
- Decision consequence reflection on significant earlier decisions.
- AI use and verification record across the lifecycle.
- Professional final product demonstration and individual engineering defence.

Assessment component	Marks	Purpose
Team product/evidence	15	Project success against stakeholders, constraints and engineering evidence.
Individual engineering defence	15	Personal competence across the lifecycle, including decision consequences and AI accountability.

21. Final Project Success Evaluation

The final defence must explicitly evaluate the project against the following dimensions:

Dimension	Question	Expected Evidence
Stakeholder value	Did the solution address the agreed problem and stakeholder needs?	Stakeholder/acceptance validation and demonstration.
Scope	Was approved scope delivered and was change controlled?	Baseline, RTM, change records, final scope comparison.
Schedule	Was work delivered within required project phases?	Milestone history, variance explanations, decisions.
Cost/resources	Were decisions consistent with cost/resource constraints?	Decision evidence, platform cost review.
Quality	Does evidence support required quality attributes?	Tests, metrics, defects, performance, acceptance results.
Security	Are security controls and residual risks understood?	Threat/security review, tests/scans, secrets handling.
Risk	Which risks materialised and how effective were mitigations?	Final Risk Register/issues/decisions.
Deployment/operations	Can the product be released, observed and recovered?	Production/staging, monitoring, rollback/recovery evidence.
Maintainability/evolution	What debt and maintenance implications remain?	Debt register, design review, evolution analysis.

22. Decision Consequence Reflection

The final project must include evidence-based reflection on several significant engineering decisions. Students should not use hindsight to label every difficult outcome a bad decision; instead, they must consider whether the original decision was reasonable given the evidence available at the time.

Original Decision →	Evidence at the Time →	Expected Benefit/Risk →	Actual Outcome →	Evidence →	Would We Decide Differently? →	Learning

Strong reflection identifies concrete engineering consequences such as:

- Rework or avoided rework
- Deployment ease/difficulty
- Integration complexity
- Quality/security impact
- Schedule or cost impact
- Technical debt
- Maintainability/changeability
- Operational consequences

23. Assessment Rules and Professional Expectations

- Evidence must be authentic and progressively produced; reconstructed evidence created immediately before assessment may receive limited or no credit.
- A working feature that violates required configuration, review or security controls may lose marks.
- Assessors may inspect live repository history and project artefacts.
- Screenshots may support evidence but do not replace stronger live/traceable evidence where available.
- Students must be able to explain the origin and purpose of every major artefact they present.
- Unsupported claims such as 'the system is secure', 'the system is scalable' or 'the platform is free' are insufficient.
- If a milestone exposes an unsuccessful engineering decision, students are expected to analyse and respond to it rather than hide it.
- Professional honesty about limitations and residual risk is valued more than unsupported claims of completeness.
- Academic and professional integrity rules apply to all human- and AI-assisted work.
- Raw milestone marks are converted to the milestone weighting specified in the project assessment architecture; raw marks must not be added directly across milestones unless the weighting has first been applied.
- Shared team-evidence marks assess the quality of the controlled team product; individual presentation/defence marks are awarded separately and may differ substantially among team members.
- Because milestone presentations contribute examination evidence, assessors may use follow-up questions to distinguish rehearsed recall from genuine understanding and individual contribution.

> The lecturer/assessor reserves the right to apply academic judgement when awarding marks where the quality, authenticity, completeness or individual ownership of the available evidence requires professional assessment. Any such judgement must remain consistent with the published assessment criteria and applicable institutional rules.

24. Project Start and Relationship to Assignment 1

Assignment 1 provides the initial research foundation for project decision-making. The Master Project Brief and Milestone 1 may be released alongside Assignment 1 so that teams can begin understanding the CivicConnect context and think ahead while completing the research task. Assignment 1 research must not be copied into the project as a

substitute for team-specific engineering evidence. Milestone 1 artefacts must be based on the registered team's CivicConnect analysis, constraints, decisions and evidence, and the formal M1 baseline is assessed separately.

25. Important Student Disclaimer

TECHNOLOGY CHOICE AND SUPPORT: Belgium Campus ITversity cannot guarantee that every programming language, framework, library, development environment, external service or deployment platform selected by a team will be installed, available, compatible with or supported on the BC Desktop platform or other institutional environment. Teams must investigate availability and compatibility before committing to a technology. Technology selection remains a team engineering decision, and the team is responsible for identifying, managing and defending the technical, schedule, cost, security and deployment risks created by that decision.

ACADEMIC INTEGRITY AND PLAGIARISM: All submitted artefacts, source code, evidence and presentations must comply with institutional academic-integrity requirements. Plagiarism, copied or misrepresented work, fabricated evidence, inappropriate reuse of another person's work, or presenting Artificial Intelligence (AI)-generated material as independently engineered work without the required verification and accountability may result in penalties under applicable institutional rules. AI is an engineering aid, not a substitute for authorship, understanding or accountability.

PRESENTATION IS COMPULSORY: Every student must participate in every assessed milestone presentation and individual engineering defence. The nature or amount of a student's contribution to the shared milestone does not remove the obligation to present and defend the engineering work. No presentation means no mark for the presentation/individual milestone assessment component, subject only to formally accepted exceptional circumstances.

ILLNESS OR OTHER ACCEPTED EXCEPTIONAL CIRCUMSTANCES: A student who is genuinely unable to present because of illness or another accepted exceptional circumstance must communicate with the lecturer and make the necessary arrangements to present at an approved later date in accordance with applicable institutional requirements. Absence does not automatically exempt a student from the presentation requirement.

Appendix A — Artefact Quality Checklist

Requirements / Scope

- Unique IDs and consistent naming.
- Clear source/stakeholder.
- Testable language.
- Acceptance criteria.
- Priority/status.
- Traceability.
- Baselined and changed only through control.

Risk Register

- Specific event/condition and cause.
- Probability and impact.

- Priority/exposure.
- Realistic mitigation and contingency.
- Owner and status.
- Updated at each milestone.
- Connected to issues/decisions where realised.

ADR / Decision Record

- Decision context.
- Relevant constraints.
- Viable alternatives.
- Decision and rationale.
- Trade-offs.
- Risks.
- Evidence.
- Later consequence updated when evidence emerges.

Pull Request / Review

- Coherent change.
- Linked issue/requirement where practical.
- Build/tests/checks visible.
- Two independent reviewers.
- Meaningful comments where needed.
- Author responds to review.
- Controlled merge.

Test / Quality Evidence

- Related requirement/risk.
- Test objective.
- Input/precondition.
- Expected result.
- Actual result.
- Pass/fail.
- Defect link if failed.
- Evidence retained.

Deployment / Operations

- Environment identified.
- Configuration/secrets controlled.

- Release version identified.
- Deployment verified.
- Rollback/recovery considered.
- Monitoring/logging evidence.
- Known limitations/residual risks.

Appendix B — Standard Presentation Marking Principles

Presentation assessment must distinguish professional communication from engineering defence. Professional communication includes structure, clarity, concise explanation, effective use of controlled evidence/visuals, delivery, timing and professional conduct. Engineering defence assesses command of evidence, application of Software Engineering principles and standards, traceability, trade-off and consequence reasoning, and individual accountability.

Dimension	High-quality performance	Weak performance
Artefact command	Navigates live artefact confidently and identifies relevant evidence.	Relies on screenshots/slides or cannot locate evidence.
SE understanding	Explains why the artefact/control matters and how it affects lifecycle outcomes.	Describes what it is without explaining purpose.
Traceability	Connects stakeholder/requirement/decision/code/test/release evidence.	Treats artefacts as disconnected documents.
Engineering judgement	Explains alternatives, trade-offs, risk and limitations.	Claims one option is 'best' without context.
Accountability	Explains personal contribution, review responsibility and AI verification.	Cannot explain own/AI-assisted work.

Appendix C — Suggested Project Folder / Repository Structure

```
/
├── README.md
├── docs/
│   ├── PED/
│   ├── requirements/
│   ├── architecture/
│   ├── decisions/
│   ├── risk/
│   ├── change/
│   ├── quality/
│   ├── security/
│   └── deployment/
├── src/
├── tests/
```

```
├── .github/
│   ├── workflows/
│   └── pull_request_template.md
├── .gitignore
└── other project-specific files
```

The structure is illustrative rather than mandatory. Teams may use another coherent structure if they can justify it and preserve traceability and maintainability.

Appendix D — Baseline Sign-Off Template

Project	CivicConnect
Baseline Type	
Version	
Date	
Scope reviewed	YES / NO
Requirements/traceability checked	YES / NO
Risk review completed	YES / NO
Repository/governance controls checked	YES / NO
Outcome	ACCEPTED / CONDITIONALLY ACCEPTED / REVISION REQUIRED

Appendix E — Change Request / Impact Analysis Template

Change ID	
Requested by	
Date	
Requested change	
Reason / expected value	
Requirements affected	
Architecture/design affected	
UI/API/data affected	
Security/privacy impact	
Quality/testing impact	

Scope impact	
Schedule/resource impact	
Cost impact	
Risk impact	
Recommendation	ACCEPT / MODIFY / DEFER / REJECT
Approval / rationale	

FINAL PROJECT QUESTION

Can the team demonstrate, with controlled artefacts, evidence and individual engineering understanding, that it engineered a successful software product rather than merely coded one?

Decision → Rationale → Implementation → Evidence → Consequence → Learning → Future Decision